**Client Profile: Beejan Technologies**

**Industry:** Telecommunications || **Business Function:** Customer Experience and Operations || **Data Sources:** Social Media, Call Centre Interaction Records, SMS, Website/App Forms

---

**Design perspective:** Most of the architectural decisions in this design are informed by my exposure to the telecommunications environment, particularly the relationship between network incidents, customer experience and operational data. I have also used relevant MTN use cases and publicly available technical references to inform assumptions where the scenario does not provide specific requirements.

## 1. The Business Scenario

Beejan Technologies receives thousands of customer complaints every day concerning issues such as poor network service, incorrect billing and poor customer service. These complaints come through social media, call-centre channels, SMS and website or application forms. The data is currently stored separately and in different formats, requiring manual consolidation into spreadsheets. This creates reporting delays, fragmented information and limited visibility across teams. The objective of the proposed pipeline is to bring these sources together, preserve the original information, process the data consistently and make it available for both near-real-time operational visibility and historical analysis.

## 2. Design Requirements

The assignment specifies seven core design considerations: data sourcing, ingestion, processing and transformation, storage, data serving, orchestration and monitoring, and DataOps. The design decisions within these areas are based on the characteristics of each source, the business use case and relevant telecommunications experience.

## 3. Proposed Solution Architecture

### 3.1 Data Source Analysis

The pipeline combines all 4 data sources. I would not treat these sources identically because their data formats, delivery patterns and operational value differ. Social-media content can include unstructured text, images and videos [1]. For the frequency, I assume that significant complaint-volume changes should become visible within **approximately 1–2 minutes**. The publicly reported MTN outages, where customers simultaneously reported problems with calls, SMS and data services, influenced this thinking [2]. Call-centre data with format of CSV and text files may include interaction records, CDR/EDR-type records, logs and transcripts [4]. For this design, I assume call-centre logs can build up over time and be processed in batches, since they do not need to be available as quickly as social-media complaints. SMS is treated as message-based data, potentially delivered as JSON/XML. I assume a **30-minute freshness target**, while website/app complaints provide structured JSON submissions and are assumed to require approximately **10-minute freshness** because they are direct customer-service requests that should be visible to the responsible team promptly, while the use case does not require second-by-second processing.

### 3.2 Ingestion Strategy

The ingestion layer will use a hybrid approach based on the requirements established above. I would use incremental ingestion for ongoing data, with a full load only for an initial historical backfill. For social media, the underlying complaint content is unstructured, but the platform API can deliver the post and its metadata as semi-structured **JSON** [1][2]. I would favour a push-based streaming approach, where the platform sends relevant posts to the pipeline as they are published. X's Filtered Stream is an example of this model [3]. Website/app submissions can similarly use push-based APIs or webhooks. Where push delivery is not available, the pipeline can pull incrementally, retrieving only new records since the previous successful extraction. SMS can use a message queue to receive and buffer incoming messages, while call-centre logs can be pulled on a schedule for batch processing. Given the possibility of significant complaint-volume increases during network incidents, the ingestion layer should also provide buffering, fault tolerance and horizontal scaling to absorb, for instance, a 10× spike without data loss or service degradation.

### 3.3 Processing & Transformation

The processing layer will use a Lambda architecture because the pipeline has both streaming and batch data. Social-media complaints require near-real-time processing, while call-centre records can be processed in batches [6]. I would first load the incoming data into the raw layer of the data lake before transforming it. This follows an ELT (Extract, Load, Transform) approach and preserves the original data for reprocessing if errors are found or transformation rules change. The data will then be cleaned, validated and standardized across sources. This includes handling missing or invalid values, standardizing timestamps and text, identifying duplicates, and normalizing text while preserving meaningful elements such as emojis, which can provide useful sentiment information. The complaints will then be classified into categories such as network, billing and customer service. NLP can support classification, while sentiment analysis can help identify customer dissatisfaction. Where available, customer and location information can be added to provide context, while severity and SLA requirements can help determine priority. The transformation produces curated data that can then be moved to analytical storage for OLAP workloads, keeping these workloads separate from operational OLTP systems.

### 3.4 Storage Architecture

The storage strategy uses a raw and curated data lake alongside a structured data warehouse. The raw layer preserves the original data from each source, maintaining data lineage and allowing the data to be reprocessed if transformation or classification rules change. Because the sources have different and potentially changing formats, the data lake will use a schema-on-read approach, allowing the data to be stored without forcing all sources into the same schema at ingestion. The operational systems are typically row-oriented, which suits OLTP workloads such as frequent inserts and updates. After ingestion, the raw data is preserved in the data lake, while curated data can be stored in Parquet, a columnar format, for efficient analytical processing. The curated data is then loaded into a structured data warehouse to support OLAP queries such as analysing complaint trends by location, category, channel and time. This approach adds some storage and management overhead, but the trade-off is flexibility and raw-data preservation in the data lake, reduced impact from changing source schemas, and efficient analytical querying in the warehouse.

### 3.5 Data Serving

The serving layer will provide different ways for downstream users to access the complaint data. Dashboards will give operations and customer-service teams visibility into current complaint volumes, emerging issues, locations and priority levels. Analysts can use SQL queries against the data warehouse for historical trends, cross-channel analysis and performance reporting. The pipeline will also provide operational alerts for significant complaint-volume spikes, high-priority complaints and potential SLA breaches. Where integration with other enterprise systems is required, the curated data can be exposed through APIs, while scheduled reports can support management and other business users.

### 3.6 Orchestration & Monitoring

The pipeline will run according to the freshness requirements defined for each source: social-media streaming will run continuously, while SMS, website/app and call-centre workloads will run at their respective processing intervals. An orchestration layer will coordinate these workflows and manage dependencies and retries. Pipeline failures will be detected through health checks, job-status monitoring and data-freshness checks. Alerts will be triggered when a job fails, data stops arriving or a defined freshness target is missed, allowing the responsible team to investigate before it affects downstream reporting.

### 3.7 DataOps

The DataOps approach will focus on making the pipeline reliable and maintainable in production. Changes to pipeline logic will be version-controlled and tested before deployment, with a rollback mechanism available to return to the last known-good version if a change causes unexpected results. This approach is informed by my exposure to telecom operations, where rollback procedures are prepared for network changes that produce unintended effects. The pipeline should also support fault recovery, horizontal scaling, access control and data-quality checks, ensuring that it can continue operating reliably as data volumes and business requirements change.

## 4. Key Assumptions & Thought Process

The following assumptions were made where the scenario does not provide specific requirements:

- **Social media:** Significant complaint-volume changes should be visible within approximately **1–2 minutes**, influenced by telecom outage scenarios and the potential for customer complaints to act as an early operational signal [2].
- **Website/App:** A **10-minute freshness target** is assumed because these are direct customer-service requests that should reach the responsible team promptly.
- **SMS:** A **30-minute freshness target** is assumed as a balance between responsiveness and processing overhead.
- **Call centre:** Records can be processed in **batch** because the primary use case is complaint analysis and reporting rather than immediate incident detection.
- **Volume:** The pipeline should be able to handle a temporary **10× increase in complaint volume** during a major service disruption. This is a stress assumption, not a claim about Beejan's actual traffic.
- **Ingestion:** Ongoing ingestion is assumed to be **incremental**, with full extraction used for an initial historical load where required.
- **Storage:** Raw data should be preserved to support **reprocessing, auditability and lineage**.
- **5. Challenges & Unknowns**
- The scenario does not specify the exact social-media platforms, API access limits, source volumes, retention requirements, customer-identification method across channels or existing SLAs. These would need to be confirmed before implementation. The structure and delivery method of call-centre CDR/EDR and transcript data would also need to be assessed, as these may differ between source systems. Finally, privacy and access requirements for customer information would need to be defined before the production pipeline is built.